

Techniques de compilation

Auditoire : CPI-2

Année Universitaire : 2025-2026

Code Classroom: **6xqrgwgl**

Dr.-Ing. Rakia Saidi

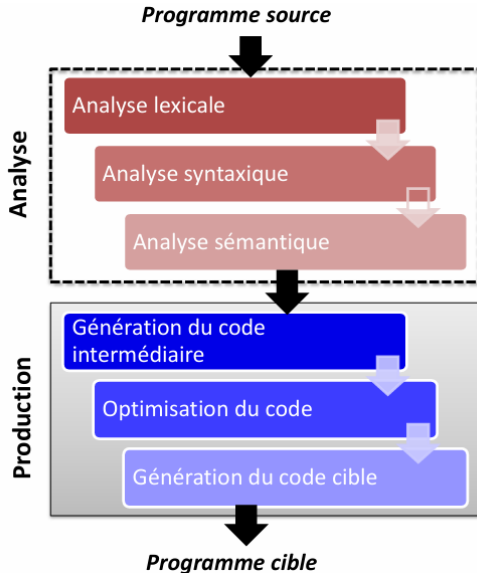
Maitre assistante en informatique

rakia.saidi@isimg.tn

Chapitre 6 :Génération de code

Plan de ce cours

- **Chapitre 1**: Introduction à la compilation
- **Chapitre 2**: Rappels à la théorie des langages
- **Chapitre 3**: Analyse lexicale
- **Chapitre 4**: Analyse syntaxique
- **Chapitre 5**: Analyse sémantique
- **Chapitre 6**: Génération de code



La **phase de production** est la deuxième grande phase de la compilation.

Rôle

Transformer la représentation intermédiaire en programme cible.

Elle comprend :

1. Génération du code intermédiaire
2. Optimisation du code
3. Génération du code cible

Représentation intermédiaire



Optimisation



Code cible (assembleur /
machine)

Le **code intermédiaire** est une représentation du programme source indépendante de l'architecture machine.

Objectifs:

- Faciliter l'optimisation du code
- Assurer la portabilité du compilateur
- Simplifier la génération du code cible

Caractéristiques

- Indépendant du processeur
- Proche du langage machine mais abstrait
- Facile à manipuler et à transformer

Prenons l'exemple:

```
a = b + c;
```

Objectif

Représenter le programme sous une forme **abstraite indépendante de la machine**

Code à trois adresses

```
t1 := b + c
```

```
a := t1
```

- t1 : variable temporaire
- Une opération par instruction
- Structure simple pour faciliter la suite (optimisation, traduction)

Objectif: améliorer performance et réduire coût (Réduire le nombre d'instructions)

Après optimisation

$a := b + c$

Explication

- Suppression de la variable temporaire t1
- Code plus direct et efficace

Allocation des registres



Objectif

Associer les variables à des registres

$b \rightarrow R1$

$c \rightarrow R2$

résultat $\rightarrow R1$

La **génération du code cible** consiste à traduire le code intermédiaire en un code exécutable par la machine (code machine ou assembleur).

Le **langage assembleur** est un langage de bas niveau :

- proche du langage machine
- dépendant du processeur
- composé d'instructions symboliques (MOV, ADD, MUL...)

MOV b, R1 ; charger b dans R1

ADD c, R1 ; $R1 = b + c$

MOV R1, a ; stocker le résultat dans a

➔ Il est ensuite traduit en **code machine binaire**.